

Uptime-Monitor – Einführung

Was hier gebaut wurde und wozu es gut ist – ohne Vorkenntnisse lesbar

Zu diesem Dokument. Es gibt zwei weitere Dokumente zu diesem Projekt: eine Projektdokumentation mit den Arbeitsschritten und eine Code-Dokumentation mit dem Programmtext. Beide setzen technisches Verständnis voraus. Dieses hier nicht. Fachbegriffe werden beim ersten Auftauchen erklärt, hinten steht zusätzlich eine Übersicht.

1. Das Problem

Eine Firma betreibt eine Website. Nachts um halb drei fällt sie aus – ein Dienst hängt, ein Zertifikat ist abgelaufen, der Server ist überlastet. Niemand merkt es.

Am nächsten Morgen um neun ruft ein Kunde an und sagt, die Seite gehe nicht. Zu diesem Zeitpunkt ist sie seit sechseinhalb Stunden weg. Kunden haben es gemerkt, bevor die Firma es gemerkt hat.

Genau diese Lücke schliesst ein Uptime-Monitor: ein Programm, das die Website in kurzen Abständen selbst aufruft und Alarm gibt, wenn sie nicht antwortet.

2. Was das Werkzeug tut

Am einfachsten stellt man es sich als jemanden vor, der eine Telefonliste abtelefoniert.

Die Liste holen

Das Programm schlägt zuerst die Liste auf – eine Datei mit den zu prüfenden Adressen, eine pro Zeile. Leere Zeilen werden dabei aussortiert. Sonst würde es später versuchen, eine leere Adresse anzurufen, und daran scheitern.

Einen Anruf machen

Dann geht es die Liste durch. Bei jedem Eintrag merkt es sich die Uhrzeit, ruft die Adresse auf und merkt sich die Uhrzeit wieder. Der Unterschied ist die gemessene Dauer.

Danach gibt es genau vier mögliche Ausgänge, und jeder wird einzeln behandelt:

- **Es hat jemand abgenommen. Das Programm notiert, was gesagt wurde: den Statuscode. 200 heisst „alles gut“, 404 heisst „die Seite gibt es nicht“.**
- **Es klingelt, aber niemand geht ran. Nach fünf Sekunden legt das Programm auf.**
- **Die Nummer existiert nicht. Die Adresse lässt sich gar nicht auflösen – als wäre der Name im Telefonbuch nicht zu finden.**
- **Die Nummer existiert, aber die Leitung ist tot. Der Rechner ist erreichbar, aber an dieser Stelle nimmt nichts entgegen.**

Warum die letzten beiden Fälle getrennt werden

Beides sieht auf den ersten Blick gleich aus: keine Antwort. Praktisch sind es zwei sehr verschiedene Probleme.

Im dritten Fall hat wahrscheinlich jemand die Adresse falsch eingetragen, oder der Namensdienst ist gestört. Im vierten Fall ist die Adresse richtig, aber der Dienst dahinter läuft nicht – abgestürzt, oder eine Firewall blockiert.

Zwei verschiedene Ursachen, zwei verschiedene Zuständige. Wer nachts um drei geweckt wird, will das sofort wissen und nicht erst suchen.

Wie das Programm den Unterschied erkennt

Wenn ein Aufruf scheitert, liefert die verwendete Programm-bibliothek **requests** eine Fehlermeldung. Darin steckt, in mehreren Schichten verpackt, die eigentliche Ursache. Das Programm packt diese Schichten auf und schaut nach, welche Art Ursache dort steht – statt sich mit „irgendwas ging nicht“ zufriedenzugeben.

Vorsichtshalber ist der Fall mitgedacht, dass die Verpackung einmal anders aussieht. Dann meldet das Programm schlicht „Verbindungsfehler“, statt selbst abzustürzen. Ein Werkzeug, das an der eigenen Fehlerbehandlung scheitert, wäre besonders ärgerlich.

Der Betrieb

Der letzte Teil ist die Wiederholung: Liste einmal holen, dann in einer Endlosschleife alle Adressen durchgehen, das Ergebnis mitschreiben und für die Auswertungssoftware bereitstellen. Danach 60 Sekunden warten und von vorne.

3. Ein Blick auf das Ergebnis

Das Programm hat gerade vier Adressen abtelefoniert. Jede Zeile ist ein Anruf.

```
19:24:06,495 INFO  https://www.srf.ch/           | Status 200           | 0.399s
19:24:06,540 ERROR http://diese-url-gibt-es-nicht.wxyz | DNS ERROR           | 0.045s
19:24:06,888 WARNING https://example.com/gibtesnicht | Status 404 (HTTP FEHLER) | 0.347s
19:24:06,890 ERROR http://127.0.0.1:9999           | CONNECTION ERROR    | 0.002s
```

Vorne steht die Uhrzeit auf die Tausendstelsekunde genau. Dann ein Wort in Grossbuchstaben – die Einstufung, wie ernst es ist. Dann die Adresse. Und hinten, nach dem Strich, wie lange der Anruf gedauert hat.

Zeile 1 – es hat jemand abgenommen

INFO heisst: alles in Ordnung, nur zur Kenntnis. Die Seite hat geantwortet, und zwar mit dem Code 200 – das ist die Rückmeldung „hier bin ich, alles gut“. Gedauert hat es vier Zehntelsekunden. Für eine Nachrichtenseite mit vielen Bildern und Skripten ist das ein normaler Wert.

Zeile 2 – die Adresse gibt es gar nicht

ERROR ist die höchste Stufe: hier kam überhaupt keine Antwort.

Der Zusatz sagt, woran es lag. Bevor ein Rechner eine Website aufrufen kann, muss er den Namen in eine Nummer übersetzen – so wie man im Telefonbuch nachschlägt. Dieser Namensdienst heisst DNS. Das Nachschlagen ist gescheitert, der Name steht nirgends.

Auffällig ist die Zeit: 0,045 Sekunden. Nicht einmal ein Zwanzigstel. Das Programm hat gar nicht erst versucht anzurufen, weil es keine Nummer hatte. Deshalb ging es so schnell.

Zeile 3 – es hat jemand abgenommen und gesagt: kenn ich nicht

WARNING ist die mittlere Stufe, und die Begründung steht in der Zeile: Es kam eine Antwort. Der Server läuft, die Verbindung stand, alles hat funktioniert – nur die angefragte Seite existiert dort nicht. Das ist der bekannte Code 404.

Warum das keine Störung im Sinne von ERROR ist: Netzwerk und Server sind in Ordnung. Das Problem liegt in der Anwendung – oder jemand hat schlicht die Adresse falsch eingetragen.

Auch hier die Zeit: 0,347 Sekunden, ähnlich wie in Zeile 1. Dieselbe Arbeit wurde gemacht. Eine Absage dauert genauso lange wie eine Zusage.

Zeile 4 – die Nummer stimmt, aber die Leitung ist tot

Wieder ERROR, aber ein anderer Grund. Diese Adresse ist der eigene Rechner. Er ist erreichbar, es gibt kein Nachschlageproblem – an der angegebenen Stelle nimmt nur nichts entgegen. Wie eine Telefonnummer, die zwar existiert, aber nicht angeschlossen ist. 0,002 Sekunden. Zwei Tausendstel, noch schneller als Zeile 2, weil der Rechner sich selbst fragt und sofort eine Absage bekommt.

Warum die Zeiten so wichtig sind

Der Vergleich von Zeile 2 und 4 zeigt: beide ERROR, beide unter einer Zwanzigstelsekunde. In beiden Fällen kam die Absage sofort. Das sagt: Es wurde gar nicht gewartet, das Problem war von Anfang an klar.

Käme dort stattdessen ein Eintrag mit 5,0 Sekunden, hiesse das etwas ganz anderes: Die Verbindung stand, das Programm hat fünf Sekunden auf eine Antwort gewartet und dann aufgegeben. Das deutet auf einen überlasteten Server hin, nicht auf einen abwesenden.

Was diese vier Zeilen zusammen leisten

Wer morgens ins Log schaut, weiss in Sekunden Bescheid:

- srf.ch läuft normal
- Eine Adresse ist falsch geschrieben oder der Namensdienst hat ein Problem
- Eine Seite fehlt, aber der Server dahinter ist gesund

- Ein Dienst antwortet nicht mehr

Und vor allem weiss er, wen er anrufen muss. Bei Zeile 2 den, der die Adressen pflegt. Bei Zeile 3 den, der die Anwendung betreut. Bei Zeile 4 den, der den Server betreibt. „Es geht nicht“ wäre keine brauchbare Information.

4. In neun Schritten entstanden

Das Projekt wurde bewusst in Stufen gebaut. Jede Stufe löst ein Problem, das die vorherige aufgeworfen hat.

Schritt 1 – Eine Adresse abfragen

Neun Zeilen Programmtext. Eine fest eingetragene Adresse aufrufen, Ergebnis auf dem Bildschirm ausgeben. Fällt die Seite aus, stürzt das Programm ab.

Schritt 2 – Adressen und Programm trennen

Die zu prüfenden Adressen stehen nun in einer eigenen Datei. Der Grund ist praktischer Natur: Wer eine Adresse ändern will, soll nicht den Programmtext anfassen müssen. Ausserdem schreibt das Programm seine Ergebnisse jetzt mit Zeitstempel mit, statt sie nur kurz anzuzeigen.

Schritt 3 – Fehler unterscheiden

Hier entsteht die Unterscheidung aus Kapitel 2. Vorher war jeder Fehler gleich, jetzt sagt das Programm, um welche Art Problem es sich handelt – zunächst in drei Fällen. Die feinere Trennung zwischen unbekannter Adresse und totem Dienst kam später dazu, als klar wurde, dass beides in einen Topf geworfen wurde.

Schritt 4 – Automatische Selbstprüfung

Das Programm bekommt Tests: kleine Zusatzprogramme, die prüfen, ob es noch richtig rechnet. Sie laufen in einer Zehntelsekunde und ohne Internetverbindung – die Antwort der Website wird dabei künstlich nachgestellt.

Der Nutzen: Wer später etwas ändert, merkt sofort, wenn er dabei etwas anderes kaputtgemacht hat. Ohne Tests fällt so etwas oft erst Wochen später auf.

Damit das überhaupt möglich war, musste das Programm umgebaut werden – von einem durchlaufenden Ablauf zu einzeln aufrufbaren Bausteinen. Dieser Umbau hat das Verhalten nicht verändert, war aber die Voraussetzung für alles Weitere.

Schritt 5 – Als Systemdienst betreiben

Bisher musste das Programm von Hand gestartet werden. Jetzt übernimmt das Betriebssystem: Es startet den Monitor beim Hochfahren, überwacht ihn und startet ihn neu, falls er abstürzt. Ein Monitoring, das man erst starten muss, ist kein Monitoring.

Schritt 6 – In einen Container packen

Ein Container ist ein Paket, das nicht nur das Programm enthält, sondern auch alles, was es zum Laufen braucht. Der bekannte Satz „auf meinem Rechner läuft es aber“ verliert damit seine Grundlage: Das Paket bringt seine Umgebung mit und verhält sich überall gleich.

Schritt 7 – Automatische Prüfung bei jeder Änderung

Bei jeder Änderung startet auf einem fremden Rechner automatisch eine Prüfung: Tests ausführen, Paket bauen. Der Fachbegriff dafür ist Pipeline.

Wichtig ist, worauf dabei geprüft wird: nicht ob es beim Entwickler läuft – das weiss er selbst. Sondern ob es auf einem Rechner läuft, der nichts über ihn weiss. Damit ist bewiesen, dass alles Notwendige tatsächlich abgelegt wurde und nichts nur lokal existiert.

Schritt 8 – Messwerte statt Textmeldungen

Bis hierhin schrieb das Programm Text, den ein Mensch lesen kann. Jetzt stellt es zusätzlich Zahlen bereit, die eine Auswertungssoftware abholen kann. Hier ist das Prometheus. Aus Zahlen lassen sich Verlaufskurven, Schwellwerte und automatische Alarmer bauen – aus Textzeilen nicht.

Beispiel: „Die Antwortzeit ist über zwei Wochen von 0,2 auf 1,8 Sekunden gestiegen“ ist eine Aussage, die man nur aus Zahlen gewinnt. Und sie warnt, bevor etwas ausfällt.

Schritt 9 – Aus Zahlen werden Kurven

Bis hierhin stellte das Programm Messwerte bereit, aber niemand holte sie ab. Jetzt kommen zwei weitere Programme dazu: eines, das die Werte in kurzen Abständen einsammelt und speichert, und eines, das sie als Kurven darstellt. Alle drei laufen zusammen als Paket und finden sich gegenseitig, ohne dass jemand Adressen eintragen muss.

Damit ist der Kreis geschlossen. Aus einer Textzeile, die ein Mensch lesen muss, ist eine Übersicht geworden, die auch dann etwas aussagt, wenn niemand hinschaut: Verfügbarkeit über die Zeit, Antwortzeiten im Verlauf, Statuscodes pro Adresse.

Dabei zeigte sich ein Fehler, der vorher nicht auffallen konnte. Fiel eine Adresse aus, blieb ihr letzter Statuscode stehen – die Auswertung zeigte weiter eine 200, obwohl seit Stunden nichts mehr antwortete. Ein alter Wert sieht aus wie ein aktueller. Das Programm entfernt den Wert jetzt ausdrücklich, statt ihn stehen zu lassen.

5. Warum so viel Aufwand für ein kleines Programm?

Das Programm selbst ist bewusst einfach gehalten. Der Aufwand steckt in dem, was darum herum liegt: Tests, Container, automatische Prüfung, Messwerte.

Das ist Absicht. In der Berufspraxis ist das Programmieren selten der schwierige Teil. Schwierig ist alles danach: Wie kommt die Software auf den Server? Wie merkt man, dass sie läuft? Was passiert, wenn jemand etwas ändert? Wer kann nachvollziehen, was wann getan wurde?

Ein bewusst kleines Programm hat den Vorteil, dass es beim Lernen nie im Weg steht. Der Blick bleibt auf dem Drumherum – und genau darum ging es.

6. Wie das in der Praxis aussieht

Ein Beispiel aus dem Betrieb zeigt den Nutzen deutlicher als das eigene kleine Projekt.

Ein IT-Dienstleister betreut 40 Kunden. Jeder hat eine Website, ein Kundenportal und einen Mailserver – zusammen rund 120 Adressen, die überwacht werden. Vertraglich hat der Dienstleister zugesagt, eine Störung innerhalb von 30 Minuten zu bemerken.

Auf einem Server liegt die Datei mit diesen Adressen. Ein neuer Kunde kommt dazu, zwei Zeilen müssen hinzugefügt werden. Dafür gibt es zwei Wege – und der Unterschied zwischen ihnen zeigt sich erst, wenn etwas schiefgeht.

Weg 1 – direkt auf dem Server

Ein Techniker verbindet sich mit dem Server, öffnet die Datei im Editor, tippt die zwei Zeilen dazu, speichert und startet den Dienst neu. Fünf Minuten, fertig. Es funktioniert: Der neue Kunde wird ab sofort überwacht.

Beim Einfügen ist allerdings eine bestehende Zeile verlorengegangen. Ein Tastendruck zu viel im Editor – die Adresse des Kundenportals von Kunde 17 ist weg.

Nichts stürzt ab. Es erscheint keine Fehlermeldung. Das Programm liest die Datei ein, findet 121 statt 122 Adressen und arbeitet sie ab. Aus seiner Sicht ist alles in Ordnung, denn es kann nicht wissen, dass dort einmal etwas anderes stand.

Drei Wochen später fällt genau dieses Portal aus. Es bleibt sechs Stunden weg, und niemand bemerkt es – bis der Kunde anruft. Also exakt die Situation aus Kapitel 1, nur diesmal mit einem Monitoring, das die ganze Zeit lief und trotzdem geschwiegen hat. Die vertraglich zugesagten 30 Minuten sind um das Zwölfwache überschritten.

Das ist der unangenehmste Zustand, den ein Überwachungssystem annehmen kann: **Ein ausbleibender Alarm sieht genauso aus wie „alles in Ordnung“**. Ein abgestürztes Monitoring fällt sofort auf. Ein Monitoring, das schweigend eine Adresse weniger prüft, fällt gar nicht auf.

Dazu kommt: Niemand kann sagen, wann die Zeile verschwunden ist und wer sie entfernt hat. Es existiert keine frühere Fassung der Datei, mit der sich vergleichen liesse.

Weg 2 – über das Repository

Die Datei wird nicht mehr auf dem Server gepflegt, sondern in einer gemeinsamen Ablage mit Versionsgeschichte – einem Repository.

Der Techniker ändert die Datei bei sich und schlägt die Änderung offiziell vor. Dabei entsteht automatisch eine Gegenüberstellung von vorher und nachher, Zeile für Zeile. Dort steht nicht „zwei Zeilen hinzugefügt“, sondern: zwei Zeilen hinzugefügt, **eine entfernt**.

Eine Kollegin liest die Änderung gegen. Die entfernte Zeile fällt ihr in derselben Sekunde auf, in der sie die Gegenüberstellung öffnet – sie muss sie nicht suchen, sie ist farblich markiert. Erst danach geht die Änderung auf den Server.

Zusätzlich läuft die automatische Prüfung aus Schritt 7 an. Sie stellt sicher, dass das Programm mit der geänderten Datei überhaupt noch startet. Was sie nicht leisten kann, ist die Frage, ob die *richtigen* Adressen drinstehen – dafür braucht es den Menschen, der gegenliest.

Im ersten Fall wird der Fehler bemerkt, nachdem er Schaden angerichtet hat, falls überhaupt. Im zweiten Fall wird er gefunden, bevor er den Server erreicht.

Der zweite Gewinn ist die Nachvollziehbarkeit. Fragt in einem Jahr jemand „Seit wann überwachen wir dieses Portal, und wer hat es eingerichtet?“, steht die Antwort in der Versionsgeschichte. Sonst liegt sie im Gedächtnis eines Kollegen, der möglicherweise nicht mehr im Unternehmen ist.

Was das mit dem kleinen Projekt zu tun hat

Dieselbe Situation trat im Projekt im Kleinen auf. Die Betriebskonfiguration des Dienstes – die Datei, mit der das Betriebssystem den Monitor startet und überwacht – lag ausschliesslich auf dem Entwicklungsrechner. Wäre der Rechner ausgefallen, hätte sie aus dem Gedächtnis rekonstruiert werden müssen.

Sie wurde daraufhin ins Repository übernommen. Derselbe Schritt wie beim Dienstleister, nur zwei Grössenordnungen kleiner – und aus demselben Grund.

Was nicht in der Ablage steht, existiert nur in jemandes Gedächtnis.

7. Begriffe

Begriff

Uptime

Statuscode

Was gemeint ist

Die Zeit, in der ein Dienst verfügbar ist.

„Uptime-Monitor“ =

Verfügbarkeitsüberwachung.

Eine Zahl, die jede Website bei einem Aufruf mitschickt. 200 heisst in Ordnung, 404 nicht gefunden, 500 Fehler auf dem

Begriff

Log

Test

Systemdienst

Container

Repository

Pipeline

DNS

Metrik

Prometheus

Grafana

Dashboard

Abholen (Scrape)

Was gemeint ist

Server.

Eine fortlaufende Mitschrift mit Zeitstempel. Wie ein Fahrtenbuch für Software.

Ein kleines Zusatzprogramm, das prüft, ob das Hauptprogramm noch richtig arbeitet.

Ein Programm, das das Betriebssystem selbst startet und überwacht – im Hintergrund, ohne dass jemand eingeloggt sein muss.

Ein Paket aus Programm und kompletter Betriebsumgebung. Läuft überall gleich.

Eine gemeinsame Ablage für Dateien mit vollständiger Versionsgeschichte. Jede Änderung bleibt nachvollziehbar.

Eine automatische Prüfstrecke, die bei jeder Änderung anspringt und Tests ausführt.

Der Namensdienst des Internets.

Übersetzt Adressen wie example.com in die Nummer, unter der der Rechner tatsächlich erreichbar ist.

Ein Messwert als Zahl, den eine Auswertungssoftware regelmässig abholt.

Verbreitete Software, die solche Messwerte einsammelt, speichert und daraus Alarme auslöst.

Software, die gesammelte Messwerte als Kurven und Übersichten darstellt.

Eine Bildschirmseite mit mehreren Anzeigen nebeneinander, die zusammen ein Lagebild ergeben.

Der regelmässige Abruf der Messwerte durch die Auswertungssoftware, hier alle 15 Sekunden.

8. Was das Projekt zeigt

- **Programmierung** – in Python, mit sauberer Behandlung von Fehlerfällen
- **Automatisierte Tests** – damit Änderungen nicht unbemerkt etwas kaputtmachen
- **Linux-Betrieb** – als Systemdienst, mit Selbstheilung nach Abstürzen
- **Containerisierung** – lauffähig unabhängig vom einzelnen Rechner
- **Automatische Prüfstrecke** – jede Änderung wird geprüft, bevor sie zählt
- **Überwachbarkeit** – Messwerte für Auswertung und Alarmierung
- **Visualisierung und Auswertung** – Messwerte werden gesammelt, gespeichert und als Verlauf dargestellt
- **Dokumentation** – damit andere den Aufbau nachvollziehen und wiederholen können